Homework 1

Fill in your name and the names of any students who helped you below.

> I affirm that I, Aashna Sibal, personally wrote the text, code, and comments in this homework assignment.
>
> I received help from [other student] who gave me suggestions on [problem].

- [Aashna Sibal]

This homework focuses on topics related to basic data types, collections, and iterations. For each problem, place your solution below the **Solution** headers. Refer to Expectations for Assignments for instructions on submitting this assignment. For this assignment, it is not necessary to write docstrings for any of your code, but you should include comments and descriptions of your general approach.

Submit this assignment on **GradeScope as a pdf (not an ipynb file).** Comments are required for problems 1,2,3, and 5. They are not required for problem 4.

Problem 1

(a)

Create a string variable `s` such that the following works:

You may need to use one or more special characters.

```
print(s)
```

```
   Tired    : Doing math on your calculator.
   Wired    : Doing math in Python.
   Inspired : Training literal pythons to carry out long division
   using an abacus.
```

In [19]:
```
# your solution

              #typed the content from above and assigned it to string s

s = """Tired    : Doing math on your calculator.
Wired    : Doing math in Python.
Inspired : Training literal pythons to carry out long division using an abacus.
"""
              #used triple quotes to make a multi line string

print(s)      #printing the string
```

```
Tired    : Doing math on your calculator.
Wired    : Doing math in Python.
Inspired : Training literal pythons to carry out long division using an abacus.
```

Next, write Python commands which use `s` to print the specified outputs. Feel free to use loops and comprehensions; however, keep your code as concise as possible. Each solution should require at most three short lines of code.

For full credit, you should minimize the use of positional indexing (e.g. `s[5:10]`) when possible.

(b)

Output:

```
Tired
Doing math on your calculator.
Wired
Doing math in Python.
Inspired
Training literal pythons to carry out long division using an
abacus.
```

In [20]:
```
# your solution here.
# we need to get everything after the colon into a new line and
# remove the : and the space after it

L=s.split(": ")    #slices the string at every occurence of ": " and makes the
                   #substrings into objects of list L

for i in L:        #iterating through the list
    print(i)       #and printing each object
```

```
Tired
Doing math on your calculator.
Wired
Doing math in Python.
Inspired
Training literal pythons to carry out long division using an abacus.
```

(c)

Output:

```
Tired
Wired
Inspired
```

Hint: look at the endings of words. A small amount of positional indexing might be handy here.

In [21]:
```
# your solution here.

for i in s.split():    #split the string at every space so each word is a
                       #different object in the list
    if i[-3:]=="red":  #index the last three characters of the object
                       #see if it equals "red"
        print(i)       #if yes, print i, the object
```

```
Tired
Wired
Inspired
```

(d)

Output:

```
Tired    : Doing math on your calculator.
Wired    : Doing math in Python.
```

Hint: These two lines are shorter than the other one.

In [22]:
```
# your solution here.

for i in s.split("\n"): #split the string at the end of each line
                        #make results objects of a list
    if len(i)<42:       #check if objects are 41 char. or less
                        #(41 is second longest line's length)
        print(i)        #print the object, i
```

```
Tired    : Doing math on your calculator.
Wired    : Doing math in Python.
```

(e)

Output (note the lack of quotation marks).

```
Tired    : Doing data science on your calculator.
Wired    : Doing data science in Python.
Inspired : Training literal pythons to carry out machine learning
using an abacus.
```

Hint: `str.replace`.

In [23]:
```
# your solution here.
# used the str.replace function two times
# one for each replacement

t=s.replace("math","data science").replace("long division","machine learning")
print(t)                          # print the string with the replacements
```

```
Tired    : Doing data science on your calculator.
Wired    : Doing data science in Python.
Inspired : Training literal pythons to carry out machine learning using an abacus.
```

Problem 2

Write code which, given a list `L`, produces a dictionary `D` where:

- The *keys* of `D` are the unique elements of `L` (i.e. each element of `L` appears only once).
- The value `D[i]` is the number of times that `i` appears in list `L`.

Demonstrate that your code works for lists of strings, lists of integers, and lists containing both strings and integers. (it's fine to copy and paste your code for this, although if you happen to know how to define functions then you might find that more efficient).

For example:

```
L = ["a", "a", "b", "c"]
# your code here
# ...
D
# output
{"a" : 2, "b" : 1, "c" : 1}
```

Solution

In [6]:
```
L = ["a", "a", "b", "c"]  #list of strings example
D = {}                    #make an empty dictionary, D

for i in L:               #i represents the objects in L
                          #use for loop to iterate through the list
    D[i]=0                #start with the dictionary key of 0,
                          #assigns objects, i, to a key in D


for i in L:
    D[i]+=1               #counting the number of times "i" is in L
                          #and making it the value of that key

print(D)                  #printing the dictionary

L = ["1","1","2","3"]     #list of integers example
D = {}

for i in L:
    D[i]=0


for i in L:
    D[i]+=1

print(D)

L = ["a","1","b","1"]     #list of strings and integers example
D = {}

for i in L:
    D[i]=0


for i in L:
    D[i]+=1

print(D)
```

```
{'a': 2, 'b': 1, 'c': 1}
{'1': 2, '2': 1, '3': 1}
{'a': 1, '1': 2, 'b': 1}
```

Problem 3

The `input()` function allows you to accept typed input from a user as a string. For example,

```
x = input("Please input a number.")
# user types 7
x
# output
'7'
```

Write code that prompts a user to input an odd `int`. If the user inputs an even `int`, the code should re-prompt them for an odd integer. After the user has entered an odd integer, the code should print a list of all numbers that the user input.

The code `int("1")` will convert strings composed of numerals into integers. You may assume that the user will only input such strings.

Hint: Try `while` and associated tools.

Example

```
# your code

> Please enter an integer.6
> Please enter an integer.8
> Please enter an integer.4
> Please enter an integer.9
> [6, 8, 4, 9]
```

Solution

In [25]:
```
x = input("Please input a number.")      #prompt the user to enter a number

Y = []                                   #empty list to add entered numbers to
Y.append(int(x))                         #convert string to int
                                         #add to list Y

while (int(x)%2)==0:                     #check if integer is even/divisble by 2
    x = input("Please input a number.") #prompt user to enter another number
    Y.append(int(x))                     #add number to list


print(Y)                                 #once user enters an odd integer,
                                         #while loop no longer applicable
                                         #print list Y
```

```
Please input a number.6
Please input a number.8
Please input a number.4
Please input a number.9
[6, 8, 4, 9]
```

Problem 4

Write list comprehensions which produce the specified list. Each list comprehension should fit on one line and be no longer than 80 characters.

In [8]:
```
# 8
```

(a)

A list of the first 10 triangular numbers. The ith triangular number is the sum of the integers up to and including i. For example, the sixth triangular number is

$$1 + 2 + 3 + 4 + 5 + 6 = 21 \ .$$

The first element in your list should be 1, and the last one should be 55.

Solution

In [26]:
```
[((i**2+i)//2) for i in range(1,11)] #use triangular numbers formula, (n^2+n)/2
                                     #for first 10 numbers, want i to go from 1
                                     #and up until, (not including) 11
```

Out[26]:
```
[1, 3, 6, 10, 15, 21, 28, 36, 45, 55]
```

(b)

A list of the letters in a predefined string `s`, **except for vowels.** For the purposes of this example, an English vowel is any of the letters `["a", "e", "i", "o", "u"]`. For example:

```
s = "make it so, number one"
# your code here
# ...
L
# output
["m", "k", "t", "s", "n", "m", "b", "r", "n"]
```

You should also exclude spaces. It is ok to include punctuation.

Hint: Consider the following code:

```
l = "a"
l not in ["e", "w"]
```

Solution

In [27]:
```
s = "make it so, number one"
[s[i] for i in range (0,(len(s))) if s[i] not in ["a","e","i","o","u"," "]]

#we want to index the string, from 0 to the end (length of s)
#if s[i] is a vowel or space (aka not in the second list), then add it to the list
```

Out[27]:
```
['m', 'k', 't', 's', ',', 'n', 'm', 'b', 'r', 'n']
```

(c)

A list `L` whose elements are themselves lists. The `i` th element of `L` contains the powers of `X[i]` from `0` to `k`, where `X` is a list and `k` an integer that can be specified by the user. For example,

```
X = [5, 6, 7]
k = 2
# your code here
# ...
L
# output
[[1, 5, 25], [1, 6, 36], [1, 7, 49]]
```

Solution

In [11]:
```
X = [5, 6, 7]
k = 2

[[(X[i]**k) for k in range(0,k+1)] for i in range(0,len(X))]

#making a new list with the object's, X[i], powers from 0 to k
#making a list of those lists ^ for every object in X, or X[0] till X[len(X)]
```

Out[11]:
```
[[1, 5, 25], [1, 6, 36], [1, 7, 49]]
```

(d)

As in **(c)**, but now include only even powers of `X`. For example,

```
X = [5, 6, 7]
k = 8
# your code here
# ...
L
# output
```

```
[[1, 25, 625, 15625, 390625],
 [1, 36, 1296, 46656, 1679616],
 [1, 49, 2401, 117649, 5764801]]
```

Solution

In [29]:
```
X = [5, 6, 7]
k = 8

[[(X[i]**k) for k in range(0,k+1) if k%2==0] for i in range(0,len(X))]


#inner list:
    #for the given object X[i] in list X, raise it to powers 0 till k
    #if k is at an even power, continue, if odd,
    #skip it and move to the next even power

#continue to do this for all objects in X upto but not including
#X[(len(X))], since the last object is X[(len(X))-1]
```

Out[29]:
```
[[1, 25, 625, 15625, 390625],
 [1, 36, 1296, 46656, 1679616],
 [1, 49, 2401, 117649, 5764801]]
```

Problem 5

In this problem, we'll simulate the *simple random walk*, perhaps the most important discrete-time stochastic process. Random walks are commonly used to model phenomena in physics, chemistry, biology, and finance. In the simple random walk, at each timestep we flip a fair coin. If heads, we move foward one step; if tails, we move backwards. Let "forwards" be represented by positive integers, and "backwards" be represented by negative integers. For example, if we are currently three steps backwards from the starting point, our position is `-3`.

Write code to simulate a random walk. Your code should:

- Specify an upper and lower bound -- once the walk reaches one of these two numbers, the process terminates.
- Keep a log of the results of the coin flips.
- Keep a log of the position of the walk at each time step.
- When the walk reaches the upper or lower bound, print a message and terminate the walk.

To simulate a coin flip, run the following code once:

```
import random
```

Then, you can let `x` be the value of a fairly flipped coin by writing:

```
x = random.choice(["heads","tails"])
```

You don't have to use `"heads"` and `"tails"` when using this function -- can you think of a more useful choice set?

Your code should include at least one instance of an `elif` statement and at least one instance of a `break` statement.

Example

```
import random

upper_bound = 3
lower_bound = -3

# your code here
# ...

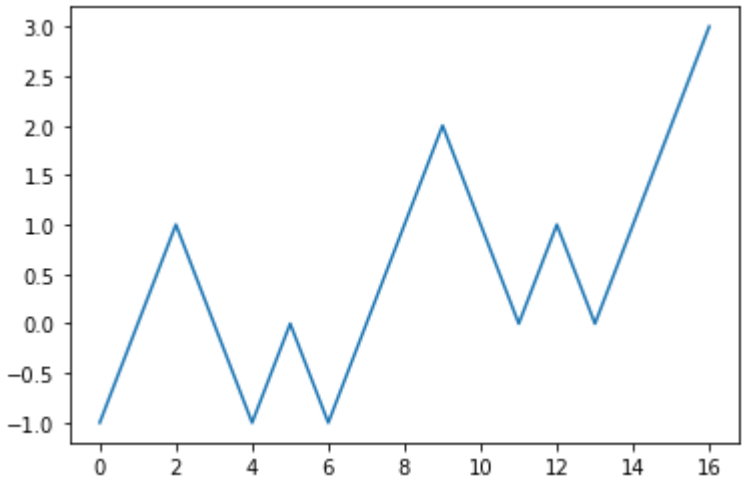
# message
Upper bound at 3 reached

positions
# output
[1, 2, 1, 2, 1, 0, 1, 2, 3]
```

Finally, you might be interested in visualizing the walk. Optionally, after your main code, run the following two lines to produce a plot. When the bounds are set very large, the resulting visualization can be quite intriguing and attractive. It is not necessary for you to understand the syntax of these commands at this stage.

In [16]:
```
from matplotlib import pyplot as plt
plt.plot(positions)
```

Out[16]:
```
[<matplotlib.lines.Line2D at 0x7f9c5900fb80>]
```



Solution

In [35]:

```
import random

upper_bound = 3   #assign upper bound to variable for consistency, to limit error
lower_bound = -3  #assign lower bound to variable for consistency, to limit error

positions = []    #empty positions list to add positions to

i = 0             #start at position 0

while True:                                          #while bounds not reached
    x = random.choice([-1,1])                        #flipping coin
    if i>= upper_bound:                              #check if position is 3
        print("Upper bound at",upper_bound,"reached") #print message
        break                                        #break loop/terminate walk
        positions.append(i)                          #record walk position

    elif i<= lower_bound:                            #check if position is -3
        print("Lower bound at",lower_bound,"reached") #print message
        break                                        #break loop/terminate walk
        positions.append(i)                          #record walk position

    if x==1:                                         #heads case
        i+=1                                         #move forward one step
        positions.append(i)                          #record walk position

    if x==-1:                                        #tails case
        i-=1                                         #move back one step
        positions.append(i)                          #record walk position

print(positions)                                     #print walk position log
```

```
Lower bound at -3 reached
[1, 0, -1, -2, -3]
```